

Informe Final

HablaLaSalle (Readuls) — Foro universitario por hilos

Tema: Aplicación Full-Stack con OAuth 2.0

Estudiantes	Rol en el proyecto	Asignatura
Nicolle Andrea Lozano Vega nlozanov@ulasalle.edu.pe	Frontend	Construcción de Software Semestre: VII Código: 3.7.3.21
Leonardo Raphael Pachari Gomez lpacharig@ulasalle.edu.pe	Base de datos	
Katherine Naomi Saico Ccahuana ksaicoc@ulasalle.edu.pe	Despliegue / Docker	
Piero Fabricio Poblete Andía ppobletea@ulasalle.edu.pe	Autenticación	
Elias Efrain Manchego Navarro emanchego@ulasalle.edu.pe	Backend	

Tipo de documento	Semestre académico	Fecha
Informe Final	2026 - I	Julio 2026

Universidad	Facultad	Carrera
Universidad La Salle	Facultad de Ingenierías y Arquitectura	Ingeniería de Software

Índice

1. Identificación del proyecto	3
2. Objetivo y alcance	3
3. Arquitectura general	4
4. Componente: Base de datos	5
4.1. Justificación del uso de dos bases de datos	5
4.2. Modelo de datos	6
4.3. Detalle de tablas y colecciones	6
5. Componente: Backend (API REST)	7
5.1. Convenciones REST y manejo de errores	7
5.2. Endpoints	8
6. Componente: Frontend	8
6.1. Páginas	9
6.2. Sesión y protección en el cliente	9
7. Componente: Autenticación y autorización (OAuth 2.0)	9
7.1. Proveedor elegido y justificación	9
7.2. Flujo Authorization Code end-to-end	9
7.3. Autorización por roles (RBAC)	10
7.4. Configuración en variables de entorno	10
8. Moderación y notificaciones	11
8.1. Ciclo de un reporte	11
8.2. Tipos de notificación	11
9. Componente: Despliegue	11
10. Entregables y cumplimiento de la rúbrica	12
11. Cómo ejecutar el proyecto	12
12. Conclusiones	13
13. Referencias	14

1. Identificación del proyecto

HablaLaSalle (Readuls) es un foro de discusión por hilos, estilo Reddit, para la comunidad de la Universidad La Salle. Los usuarios inician sesión con su correo institucional (`@ulasalle.edu.pe`) mediante **OAuth 2.0 Authorization Code delegado a Auth0**, se organizan en comunidades con distintos niveles de privacidad, publican con etiquetas (**HELP**, **ANNOUNCEMENT**, **DISCUSSION**, **CASE**) y comentan en hilos anidados de profundidad arbitraria. El sistema incluye votación, reportes de contenido, un panel de moderación con rol delegado a Auth0 (RBAC) y un sistema de notificaciones de extremo a extremo.

Accesos del proyecto	
Repositorio Git	https://github.com/p-poblete/foro-uls
Frontend (producción)	https://foro-uls.readuls.workers.dev
API (producción)	https://readuls-843408448797.us-central1.run.app
Colección Postman	postman_collection.json (raíz del repositorio)
Variables de entorno	backend/.env.example y frontend/.env.example
Documentación	README.md (raíz) + README por módulo (backend/, frontend/, bd/)

2. Objetivo y alcance

Diseñar e implementar una aplicación web completa (backend + frontend) que integre: una API REST, dos bases de datos (relacional y NoSQL) con uso justificado, un frontend moderno con SSR, y autenticación/autorización con **OAuth 2.0 Authorization Code** delegada a un proveedor de terceros (Auth0), validando tokens en cada petición protegida y garantizando que cada usuario solo pueda operar sobre los datos que le pertenecen.

Funcionalidades implementadas:

- Login/logout institucional vía Auth0 (Google detrás), restringido al dominio `@ulasalle.edu.pe`.
- Perfiles de usuario: avatar, biografía, carrera, configuración propia protegida.
- Comunidades con privacidad configurable: *pública* (ingreso directo), *restringida* (solicitud con aprobación del dueño) y *privada* (solo invitación); membresías, listado de miembros, expulsión.
- Publicaciones con etiqueta, imagen y enlaces; votación (upvote/downvote, un voto por usuario); comentarios anidados sin límite de profundidad.
- Reportes de contenido y **moderación real**: banear (eliminar), mantener o descartar con motivo; suspensión/archivado de comunidades; desactivación de cuentas.
- Notificaciones para todas las acciones relevantes (votos, comentarios, respuestas, anuncios, ciclo completo del reporte), con anonimato para proteger a quien reporta.
- Subida de imágenes a almacenamiento de objetos S3-compatible.

3. Arquitectura general

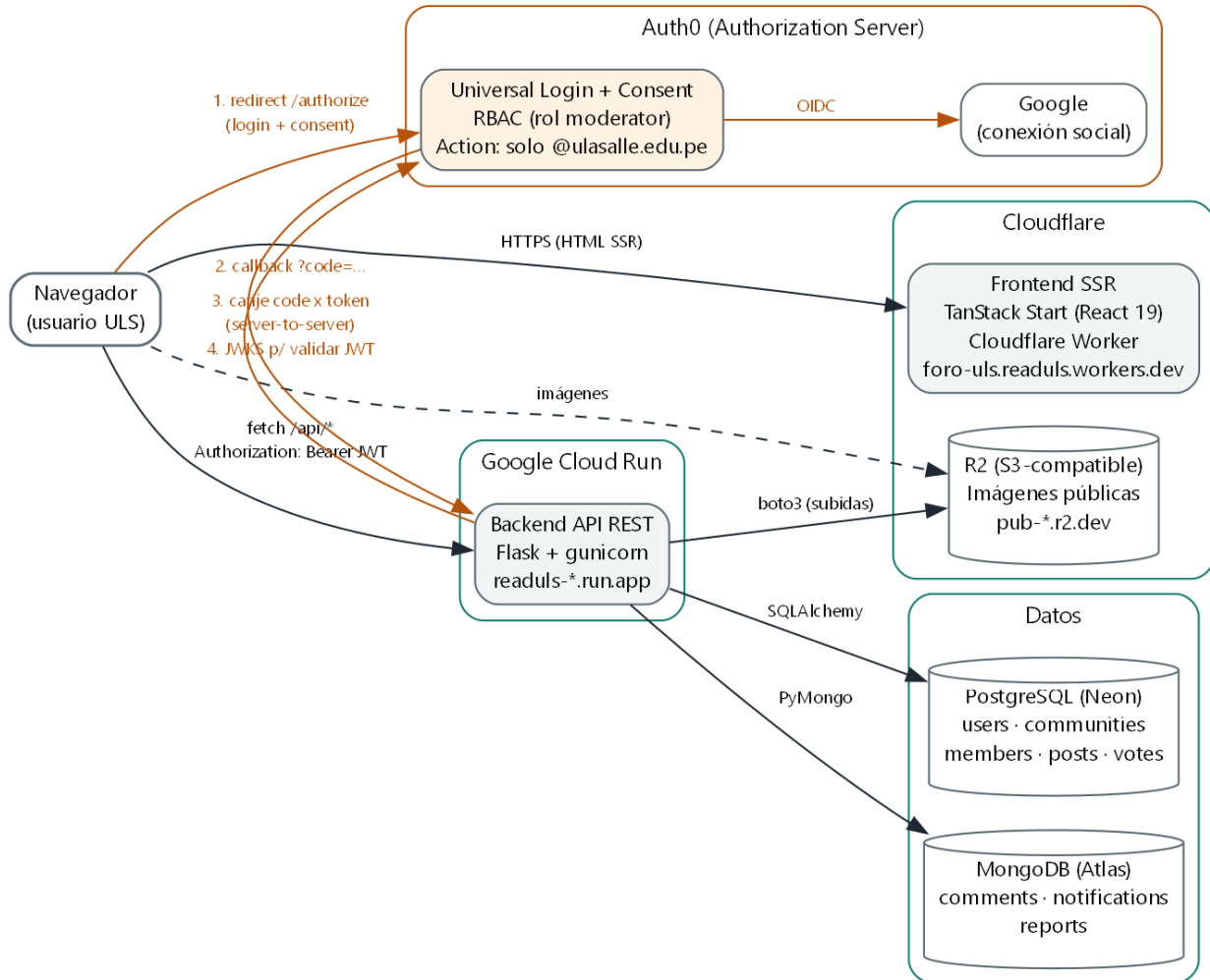


Figura 1: Arquitectura de componentes y flujo OAuth 2.0 (en naranja).

Capa	Tecnología	Hosting
Frontend	TanStack Start (React 19, SSR) + Tailwind CSS v4 + shadcn/ui, empaquetado con Bun/Vite	Cloudflare Workers
Backend	Flask 3 (API REST) + SQLAlchemy + PyMongo, servido con gunicorn	Google Cloud Run
Autenticación	Auth0 — OAuth 2.0 Authorization Code + OIDC, RBAC	Auth0 (SaaS)
BD relacional	PostgreSQL	Neon
BD NoSQL	MongoDB	MongoDB Atlas
Objetos	Cloudflare R2 (S3-compatible, boto3)	Cloudflare

Tabla 1: Stack tecnológico y plataformas de despliegue.

4. Componente: Base de datos

4.1. Justificación del uso de dos bases de datos

- **PostgreSQL (relacional):** usuarios, carreras, comunidades, membresías, publicaciones y votos. Son datos estructurados con relaciones fuertes que exigen **integridad referencial**: un voto no puede existir sin su publicación y su usuario (**FOREIGN KEY + UNIQUE(post_id, user_id)**), una membresía es única por par comunidad-usuario, y las consultas del feed usan *joins*.
- **MongoDB (NoSQL):** comentarios, notificaciones y reportes. Son **documentos flexibles de alta escritura**: los comentarios forman un árbol de profundidad arbitraria (*materialized path* vía **parent_id + depth**) que en SQL exigiría *joins* recursivos costosos; las notificaciones y reportes tienen forma variable según el tipo y no necesitan transaccionalidad relacional.

4.2. Modelo de datos

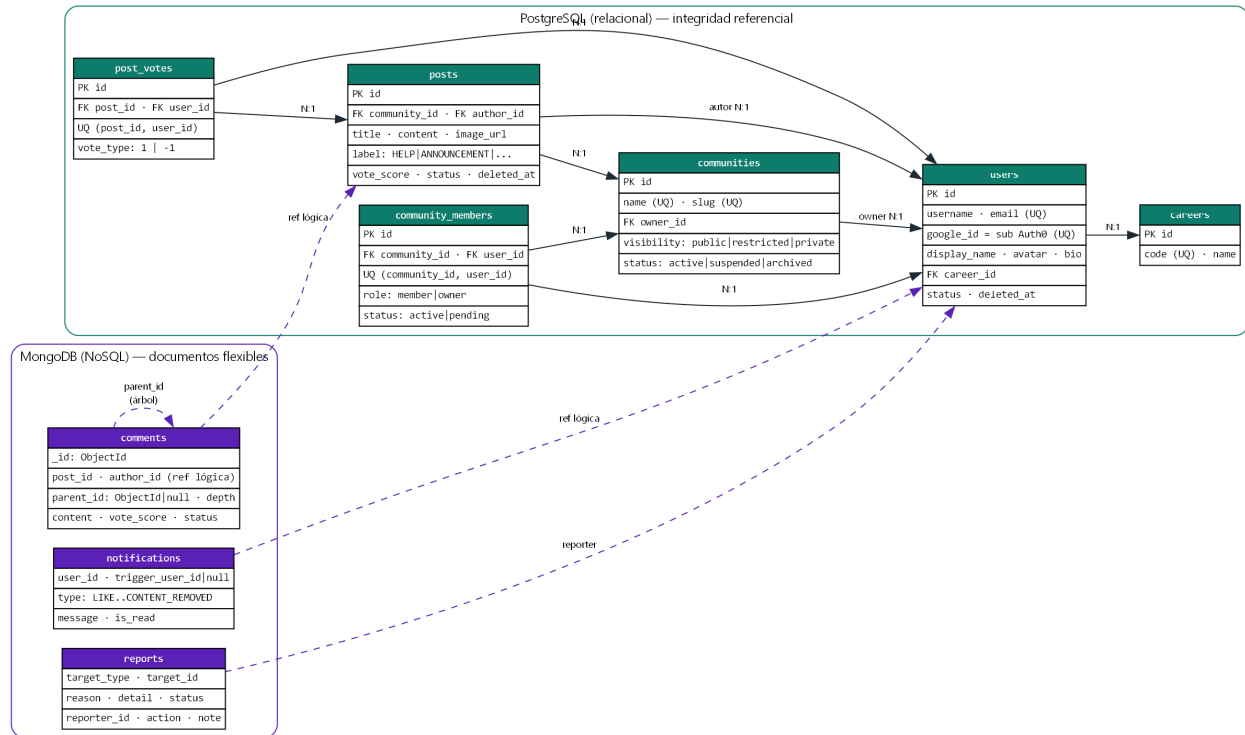


Figura 2: Modelo entidad-relación: tablas PostgreSQL (izquierda/arriba) y colecciones MongoDB con referencias lógicas (punteadas).

4.3. Detalle de tablas y colecciones

PostgreSQL (creadas por SQLAlchemy con `db.create_all()` a partir de `backend/models.py`):

Tabla	Contenido y restricciones
users	Perfil; <code>google_id</code> guarda el sub de Auth0 (único); <i>soft delete</i> con <code>deleted_at</code> .
careers	Catálogo de carreras ULS (<code>code</code> único).
communities	<code>visibility</code> : public/restricted/private; <code>status</code> : active/suspended/archived; dueño con FK.
community_members	N:M usuario-comunidad; <code>UNIQUE(community_id, user_id)</code> ; <code>status</code> active/pending para el flujo de aprobación.
posts	Publicación con etiqueta, imagen, <code>vote_score</code> desnormalizado y <i>soft delete</i> .
post_votes	Un voto por usuario y publicación (<code>UNIQUE</code>); <code>vote_type</code> ∈ {1, -1}.

MongoDB (índices creados al arrancar, `backend/database.py`):

Colección	Contenido	Índices
comments	Árbol por <code>parent_id + depth</code> ; votos con <code>\$inc</code> atómico; <i>soft delete</i> .	(<code>post_id</code> , <code>parent_id</code>), (<code>post_id</code> , <code>created_at</code>)
notifications	10 tipos; <code>trigger_user_id = null</code> para notificaciones anónimas de moderación.	(<code>user_id</code> , <code>created_at</code>)
reports	Reporte + resolución (<code>status</code> , <code>action</code> , <code>note</code> , <code>resolved_by</code>).	(<code>status</code> , <code>created_at</code>)

5. Componente: Backend (API REST)

API Flask organizada en *blueprints* por dominio (`backend/routes/`): `auth`, `users`, `communities`, `posts`, `comments`, `reports`, `careers`, `uploads`.

5.1. Convenciones REST y manejo de errores

- Códigos correctos: 200 lectura, 201 creación, 204 borrado, 400 validación, 401 sin token, 403 sin permiso, 404 no encontrado, 409 conflicto.
- Formato de error único en toda la API, incluidos los manejadores globales de 404/405/500 (nunca HTML de Flask):

Listing 1: Shape de error consistente

```
{ "error": { "code": "FORBIDDEN",  
            "message": "No autorizado: no eres el dueño de este recurso" } }
```

5.2. Endpoints

Endpoint	Acceso	Descripción
GET /api/auth/login	Público	Inicia el flujo Authorization Code
GET /oauth/callback	Auth0	Canjea el <code>code</code> , provisiona usuario
GET /api/auth/me	Bearer	Usuario autenticado (valida JWT)
GET /api/auth/logout	Público	Cierra sesión SSO en Auth0
GET /api/users, /api/users/:id	Público	Listado y perfil
PUT DELETE /api/users/:id	Dueño*	Editar / desactivar cuenta
GET /api/users/:id/notifications	Dueño	Notificaciones propias
PATCH /api/users/:id/notifications/read	Dueño	Marcar todas leídas
GET /api/communities, /:id	Público	Con <code>member_count</code> y membresía
POST /api/communities	Bearer	Crear (privacidad configurable)
PUT DELETE /api/communities/:id	Dueño*	Editar / archivar
POST /api/communities/:id/join leave	Bearer	Unirse (según privacidad) / salir
GET /api/communities/:id/members	Público	Pendientes solo para el dueño
PATCH DELETE .../members/:user_id	Dueño	Aprobar / expulsar
GET /api/posts, /:id, .../posts	Público	Feed global, por comunidad, detalle
POST /api/posts	Bearer	Crear (autor = usuario del token)
PUT DELETE /api/posts/:id	Dueño*	Editar / eliminar
POST /api/posts/:id/vote	Bearer	Votar (toggle, un voto por usuario)
GET POST /api/posts/:id/comments	Púb./Bearer	Hilo de comentarios / comentar
PUT DELETE /api/comments/:id	Dueño*	Editar / eliminar comentario
POST /api/comments/:id/vote	Bearer	Votar comentario
POST /api/reports	Bearer	Reportar contenido
GET /api/reports	Moderador	Bandeja con reportante visible
PATCH /api/reports/:id	Moderador	Resolver: banear/mantener/descartar + motivo
GET /api/careers	Público	Catálogo
POST /api/uploads	Bearer	Subir imagen, devuelve URL pública

Tabla 2: Endpoints de la API. * = el moderador (rol Auth0) también puede.

6. Componente: Frontend

SPA con **Server-Side Rendering** (TanStack Start / React 19) desplegada como Cloudflare Worker: el servidor renderiza el *shell* completo de la aplicación y la metadata SEO/OpenGraph en cada petición; los datos de la API se cargan en el cliente con TanStack Query tras la hidratación.

6.1. Páginas

Ruta	Página
/login, /oauth/callback	Login (botón único Auth0) y recepción del token
/, /feed/label/:label, /feed/career/:code, /feed/announcements	Feed global y filtrado por etiqueta, carrera o anuncios
/communities, /communities/create	Listado y creación con selector de privacidad
/communities/:id, /:id/members, /:id/edit	Detalle con unirse/salir, miembros con aprobación, edición
/publications/:id, /:id/edit	Detalle con hilo de comentarios, votos, reporte; edición
/users/:id, /users/:id/settings	Perfil público; configuración solo del propio usuario
/notifications	Campana: sociales (con avatar) y de moderación (anónimas)
/moderation	Panel de moderación (solo rol moderator)
/bookmarks, /search, /rules, /onboarding	Guardados, búsqueda, reglas, primer inicio

Tabla 3: Rutas del frontend (TanStack Router, file-based).

6.2. Sesión y protección en el cliente

- El cliente HTTP central adjunta **Authorization: Bearer <access_token>** en cada petición; un 401 limpia la sesión local.
- *Guards* de ruta: **requireAuth** (exige sesión) y **requireSelf** (rutas personales: cambiar el id en la URL redirige a la propia), con re-verificación dentro del componente como defensa en profundidad.
- **isModerator()** decodifica el claim de roles del JWT solo para decidir qué UI mostrar; la autorización real siempre la aplica el backend.
- La interfaz refleja **login/logout** y muestra solo los datos del usuario autenticado (notificaciones, configuración, acciones de dueño).

7. Componente: Autenticación y autorización (OAuth 2.0)

7.1. Proveedor elegido y justificación

Se eligió **Auth0** como Authorization Server de terceros porque: (a) implementa OAuth 2.0/OIDC estándar con Universal Login y pantalla de consentimiento hospedadas; (b) permite mantener el acceso con Google institucional como conexión social *detrás* del proveedor, con una *Action* que rechaza cuentas fuera de @ulasalle.edu.pe; (c) provee **RBAC** para delegar también la autorización (rol **moderator** inyectado en el token); y (d) su capa gratuita cubre el alcance del proyecto sin tarjeta de crédito.

7.2. Flujo Authorization Code end-to-end

1. El usuario pulsa *Iniciar sesión* en el frontend → GET /api/auth/login redirige a [https://{tenant}.auth0.com/authorize?response_type=code,client_id,redirect_uri,scope=openid profile email,audience y state](https://{tenant}.auth0.com/authorize?response_type=code&client_id=&redirect_uri=&scope=openid%20profile%20email&audience=&state=) (anti-CSRF).
2. Auth0 muestra login + consentimiento (Google institucional detrás); la Action *post-login* bloquea dominios no institucionales.
3. Auth0 redirige al backend (/oauth/callback) con el **code**; el backend verifica el **state** y canjea el código **server-to-server** en /oauth/token usando el **client_secret** (nunca expuesto al navegador).

4. El backend recibe el `access_token` (JWT RS256), provisiona el usuario local por su `sub` y lo entrega al frontend en el *fragmento* de la URL (que no viaja a servidores y se elimina de la barra de direcciones inmediatamente).
5. En cada petición protegida, el backend valida el JWT contra el **JWKS** del tenant (llaves públicas cacheadas): firma, `aud` (= `AUTH0_AUDIENCE`), `iss` y `exp`.
6. **Ownership por sub**: la identidad se resuelve únicamente desde el token validado; el autor de cada recurso creado es el usuario del token (se ignora cualquier id enviado por el cliente), y editar/borrar recursos ajenos responde 403.
7. **Logout** completo: limpia la sesión local y cierra la sesión SSO en Auth0 (`/v2/logout`), permitiendo reintentar con otra cuenta (también tras un rechazo por dominio).

Listing 2: Validación del access token por JWKS (backend/auth_utils.py)

```
_jwk_client = PyJWKClient(f"https://{AUTH0_DOMAIN}/.well-known/jwks.json")

def verify_access_token(token):
    signing_key = _jwk_client.get_signing_key_from_jwt(token).key
    return jwt.decode(token, signing_key, algorithms=["RS256"],
                      audience=AUTH0_AUDIENCE, issuer=_ISSUER)

def current_user():
    claims = verify_access_token(_bearer_token())
    return User.query.filter_by(google_id=claims["sub"],
                              deleted_at=None).first()
```

7.3. Autorización por roles (RBAC)

El rol `moderator` se administra en Auth0 (User Management → Roles) y una Action lo inyecta en el token como claim *namespaced* (`https://readuls/roles`). El decorador `@require_moderator` protege la bandeja de reportes y las acciones de moderación; `forbid_unless_owner_or_moderator` permite al moderador retirar o editar contenido ajeno (siempre notificando al autor).

7.4. Configuración en variables de entorno

Ninguna credencial está *hardcodeada*; todo vive en `.env` (ignorado por git) con plantillas `.env.example` versionadas:

Variable	Uso
<code>AUTH0_DOMAIN</code>	Tenant del Authorization Server
<code>AUTH0_CLIENT_ID</code> / <code>_SECRET</code>	Credenciales del cliente OIDC (canje server-to-server)
<code>AUTH0_AUDIENCE</code>	Identificador de la API en Auth0 (hace que el token sea JWT)
<code>AUTH0_CALLBACK_URL</code>	Redirect URI registrada en Auth0
<code>AUTH0_ROLES_CLAIM</code>	Claim namespaced de roles (RBAC)
<code>POSTGRES_URL</code> / <code>MONGO_URL</code>	Conexiones a Neon y Atlas
<code>AWS_*</code>	Credenciales y endpoint del almacenamiento S3-compatible
<code>FLASK_SECRET_KEY</code> / <code>FRONTEND_URL</code>	Sesión de Flask (state OAuth) y retorno al frontend

Tabla 4: Variables de entorno principales (ver backend/.env.example).

8. Moderación y notificaciones

8.1. Ciclo de un reporte

1. Un usuario reporta una publicación o comentario (motivo + detalle).
2. El autor del contenido recibe una notificación **anónima**: «*Alguien ha reportado tu publicación por el motivo ...*» — nunca se revela quién reportó.
3. El moderador ve el reporte en el panel (con el nombre del reportante) y lo resuelve con motivo adjunto: **eliminar** el contenido (*soft delete*), **mantener** o **descartar**.
4. Reportante y autor reciben el veredicto con el motivo del moderador.

8.2. Tipos de notificación

Tipo	Se dispara cuando	Destinatario	Anónima
LIKE / DISLIKE	Votan tu publicación	Autor	No
COMMENT	Comentan tu publicación	Autor del post	No
REPLY	Responden tu comentario	Autor del padre	No
COMMUNITY_POST	Nueva publicación en tu comunidad	Miembros	No
ANNOUNCEMENT	Anuncio en tu comunidad	Miembros	No
REPORT_RECEIVED	Reportan tu contenido	Autor	Sí
REPORT_RESOLVED	Se resuelve un reporte	Reportante y autor	Parcial
CONTENT_REMOVED	Un moderador banea tu contenido	Autor	Sí
CONTENT_EDITED	Un moderador edita tu contenido	Autor	Sí

Tabla 5: Cobertura del sistema de notificaciones.

9. Componente: Despliegue

- **Backend** en Google Cloud Run: despliegue desde código fuente (`gcloud run deploy readuls --source backend`); Cloud Build construye el `Dockerfile` (gunicorn, `$PORT` inyectado). Escala a cero (arranque en frío de 1–3 s).
- **Frontend** en Cloudflare Workers: `bun run build + wrangler deploy`. La URL de la API se fija en tiempo de build (`VITE_API_BASE_URL`).
- **Datos gestionados**: Neon (PostgreSQL, TLS obligatorio), MongoDB Atlas (M0, cadena SRV) y Cloudflare R2 (imágenes públicas).
- **Desarrollo local**: `docker compose up` levanta Postgres, MongoDB, MinIO (S3 local), backend y frontend.
- Guía completa paso a paso en `docs/DEPLOY.md`.

Listing 3: Comandos de despliegue

```
# Backend -> Cloud Run (construye el Dockerfile con Cloud Build)
gcloud run deploy readuls --source backend --region us-central1 \
  --env-vars-file env.yaml

# Frontend -> Cloudflare Workers
cd frontend && bun run build
bunx wrangler deploy -c .output/server/wrangler.json --name foro-uls
```

10. Entregables y cumplimiento de la rúbrica

Entregable	Evidencia
Repositorio Git	https://github.com/p-poblete/foro-uls — backend + frontend + bd + docs, con historial de commits del grupo.
README.md	Tema, arquitectura, proveedor OAuth y justificación, justificación de las 2 BD, cómo correr todo; README técnico adicional por módulo.
.env.example	backend/.env.example y frontend/.env.example con todas las variables y placeholders.
Postman	postman_collection.json: carpetas <i>Público</i> , <i>Autenticado (Bearer)</i> , <i>Moderador</i> y <i>Errores esperados</i> (401/403/404), con variables <code>base.url</code> y <code>token</code> .
Diagrama	Figuras 1 (arquitectura + flujo OAuth) y 2 (modelo de datos).
Demo en vivo	https://foro-uls.readuls.workers.dev (frontend) y https://readuls-843408448797.us-central1.run.app (API).

Tabla 6: Entregables del trabajo final.

Criterio	Pts	Cómo se cumple
Backend y API	4	API REST funcional en producción; códigos HTTP correctos (200/201/204/400/401/403/404/409); formato de error único + manejadores globales.
Bases de datos	3	PostgreSQL y MongoDB en uso efectivo con modelo coherente y justificación documentada (sección 4 y README).
OAuth Authorization Code	5	Flujo completo end-to-end con Auth0; canje server-to-server; validación JWT por JWKS en cada request protegido; ownership por <code>sub</code> ; config 100 % en <code>.env</code> .
Frontend	3	SSR con TanStack Start; consume la API; login/logout reflejados; solo datos del usuario autenticado (guards + backend).
Colección Postman	2	Todos los requests públicos y protegidos, incluyendo casos 401/403/404.
Documentación	1	README ejecutable + <code>.env.example</code> + diagramas + guía de despliegue.
Código y Git	1	Estructura modular por dominio; commits descriptivos distribuidos en el grupo.
Presentación	1	Aplicación corriendo en vivo en las URLs indicadas.

Tabla 7: Trazabilidad contra la rúbrica de evaluación (20 pts).

11. Cómo ejecutar el proyecto

Listing 4: Arranque local con Docker

```
cp backend/.env.example backend/.env # completar AUTH0_* y FLASK_SECRET_KEY
cp frontend/.env.example frontend/.env
docker compose up -d --build
# Frontend: http://localhost:8080 | Backend: http://localhost:5000
```

Para probar la API sin frontend: importar `postman_collection.json`, iniciar sesión en la app y copiar el token `auth.token` desde el `localStorage` del navegador a la variable `token` de Postman.

12. Conclusiones

- Delegar la autenticación a un Authorization Server de terceros (Auth0) simplificó el sistema y elevó su seguridad: la aplicación nunca maneja contraseñas, y la validación por JWKS ancla cada petición a una identidad verificada criptográficamente.
- La separación relacional/NoSQL respondió a criterios técnicos verificables: integridad referencial para el dominio estructurado y documentos flexibles para árboles de comentarios, notificaciones y reportes de alta escritura.
- El *ownership* por `sub` y el RBAC delegado demostraron que autorización y autenticación pueden externalizarse sin perder control fino en la API (401/403 correctos y verificables en Postman).
- El despliegue en capas gratuitas gestionadas (Cloud Run, Cloudflare, Neon, Atlas) permitió operar una aplicación completa en producción sin costo, con la única concesión del arranque en frío.

13. Referencias

- <https://auth0.com/docs/get-started/authentication-and-authorization-flow/authorization-code-flow>
- <https://datatracker.ietf.org/doc/html/rfc6749> — The OAuth 2.0 Authorization Framework
- <https://flask.palletsprojects.com/>
- <https://tanstack.com/start>
- <https://www.mongodb.com/docs/manual/applications/data-models-tree-structures/>
- <https://cloud.google.com/run/docs>
- <https://developers.cloudflare.com/workers/>